Reorganizing cuADMM

An ADMM-first, cone-extensible architecture with pluggable PSD-projection strategies and cuDSS / CHOLMOD linear-system backends

Reorganization plan for the cuADMM developers

October 13, 2026

Baseline: `main` at `9bb4ba5` (submodule `psd_projection` at `b566590`), plus the uncommitted penalty-rule edit of October 13, 2026 (Section 3.5)

Abstract

cuADMM is a GPU solver for large multi-block conic programs, built around the first-order method of [1] and the factorization-free PSD projection of [2]. Its numerics are sound and well tested, but nearly all logic lives in one class (`SDPSolver`) whose `solve()` method spans about 1,080 lines, configuration is spread over positional arguments, public fields and preprocessor constants, and several algorithmic options are either compiled out or unreachable. This document is a complete plan to reorganize the code into a layered library in which (i) plain ADMM with step length $\tau \in (0, 2)$ is the default algorithm and sGS-ADMM is a runtime option, justified by the equivalence result of [3]; (ii) every cone is a plug-in behind one interface, so that second-order and other cones can be added without touching the solver; (iii) the PSD projection is a catalog of explicit, named strategies—including the composite polynomial filter of [2], integrated directly instead of through a submodule—selected by a documented policy; and (iv) the normal-equation solve is an interface with a cuDSS backend first and a CHOLMOD backend second. The plan lists the defects found during the review, the target architecture with interface sketches and naming conventions, a single table of options and defaults, a test strategy, and an incremental migration in ten phases, each with acceptance criteria.

Contents

1 Summary **1**

2 Background and the three papers **2**

3 Mathematical specification **3**

3.1 Problem class . . . 3

3.2 ADMM, sGS-ADMM and their equivalence . . . 3

3.3 Cone step for a product cone . . . 4

3.4 Scaling and KKT residuals . . . 4

3.5 Penalty update (current default) . . . 4

4 Assessment of the current code **5**

4.1 What must be preserved . . . 5

4.2 Structural problems . . . 5
4.3 Defect register . . . 6

5 Design principles **7**

6 Target architecture **7**
6.1 Directory layout . . . 7
6.2 Naming and coding conventions . . . 9
6.3 Public API . . . 9

7 Algorithm layer **10**
7.1 The iteration loop . . . 10
7.2 Iteration schemes . . . 10
7.3 Penalty policies . . . 10
7.4 Termination and validation . . . 11
7.5 Reporting . . . 11

8 Cones **11**
8.1 Interface . . . 11
8.2 Existing cones . . . 12
8.3 Adding a cone (checklist) . . . 12
8.4 Planned cones . . . 12

9 PSD cone projection strategies **12**
9.1 Organization . . . 12
9.2 Strategy catalog . . . 13
9.3 Selection policy . . . 13
9.4 Integrating the composite polynomial filter . . . 14
9.5 LOBPCG improvements . . . 15
9.6 PSD options . . . 16

10 Normal-equation solvers **16**
10.1 Role . . . 16
10.2 Interface . . . 16
10.3 Backends . . . 17
10.4 Selection . . . 17
10.5 Rank deficiency and presolve . . . 17
10.6 Benchmark protocol . . . 17

11 Scaling **18**

12 Core infrastructure **18**

13 Unified options 18

14 Testing, benchmarking and continuous integration 19

15 Migration plan 19

16 Open decisions 21

A Mapping from current files to the new layout 21

B Notation and code names 23

1 Summary

Decisions already taken.

D1. ADMM is the default algorithm. The default iteration is plain (two-block) ADMM with multiplier step length $\tau = 1.618$; $\tau = 1.918$ is offered as the aggressive preset, and any $\tau \in (0, 2)$ is accepted. sGS-ADMM becomes a *runtime* option (today it is removed at compile time by `CUADMM_PLAIN_ADMM_ONLY`). Rationale: ADMM and sGS-ADMM share the same fixed-point map in the z-variable, and every $\tau \in (0, 2)$ is admissible [3, 8, 7] (Section 3.2).

D2. Linear systems: cuDSS first, CHOLMOD second. The normal-equation solver is an interface with two production backends: cuDSS (GPU factorization and GPU solves, preferred when available) and CHOLMOD (CPU factorization, CPU or dense-GPU solves, always available). A dense cuSOLVER backend is optional (Section 10).

D3. Composite polynomial filtering is integrated directly. The kernels of [2] move into cuADMM as one PSD projection strategy among several; the `psd_projection` submodule is retired (Section 9).

D4. Cones are plug-ins. Free, nonnegative and PSD blocks are ported to a common `Cone` interface; second-order cones are the first new cone (Section 8).

D5. Software-engineering standards. One options object with one table of defaults, explicit ownership of GPU resources, no allocation inside the iteration loop, deterministic by default, consistent naming, and a behavior-preserving migration in which every structural step reproduces the current iterates bit for bit (Sections 5–15).

What the reader should take away. Table 1 summarizes the migration. Phases 0–6 restructure the code without changing its iterates (except for explicit bug fixes); Phases 7–9 change behavior on purpose and are validated against tolerances instead of bitwise histories.

2 Background and the three papers

- **cuADMM** [1] introduced the solver: sGS-ADMM for multi-block SDPs, PSD projection by cuSOLVER (batched Jacobi for many small blocks, QR-based EVD on streams for larger ones), and a hybrid CPU–GPU normal-equation solve with CHOLMOD, building on the trajectory

Phase	Content	Iterates
0	Safety net: frozen baseline histories, CI build matrix, environment fixes	unchanged
1	Cleanup: dead code, compile-time algorithm flag, the defects of Table 2 that are one-line fixes	unchanged[†]
2	`SolverOptions`: one source of defaults; CLI and MATLAB rebuilt on it	unchanged
3	Core infrastructure: `GpuContext`, stream pool, workspace arena	unchanged
4	Algorithm layer: `IterationScheme`, `PenaltyPolicy`, termination, observers	unchanged
5	Cone layer: `Cone` interface; free, nonnegative and PSD cones ported	unchanged
6	PSD strategies and composite filter integrated; submodule retired	unchanged[‡]
7	Linear systems: `NormalEquationSolver`, CHOLMOD port, cuDSS backend, presolve	within tolerance
8	Data layout: variables grouped by cone, concurrent cone projections	within tolerance
9	New cones: second-order and rotated second-order	new tests
10	Documentation, examples, benchmarks, release	—

Table 1: Migration phases (details in Section 15). [†]Except the stream race fix (B2), which only removes a possible nondeterminism. [‡]For the exact-EVD paths; the composite path changes because of the math-mode fix (B1).

optimization pipeline of [4]. Its experiments target sparse moment relaxations of contact-rich planning and multi-block SDPs from Mittelmann's collection.

- **Composite polynomial filtering** [2] replaces the eigenvalue decomposition of a large dense block by a short composition of odd degree-5 polynomials evaluated with GEMMs in FP16 or FP32, after scaling the spectrum with a Lanczos bound. It is designed as a low-precision *warm start* inside ADMM, followed by FP64 projections, and notes that ADMM quickly enters a low-rank regime [5] in which LOBPCG becomes attractive.
- **Afterimage slow regions** [3] studies first-order methods for linear conic programs as fixed-point iterations. Its Table 1 and Appendix A.2 show that ADMM and sGS-ADMM induce the *same* fixed-point map in the auxiliary variable z, which is the basis of decision D1.

3 Mathematical specification

This section fixes the mathematics the code must implement; class and function names in later sections refer to the symbols introduced here (Appendix B).

3.1 Problem class

Let $\mathbb{X} = \mathbb{X}_1 \times \cdots \times \mathbb{X}_B$ and $\mathcal{K} = \mathcal{K}_1 \times \cdots \times \mathcal{K}_B$ with each $\mathcal{K}_\beta$ a closed convex cone in $\mathbb{X}_\beta$. cuADMM solves the primal–dual pair

$$\min_{X \in \mathbb{X}} \langle C, X \rangle \ \text{ s.t. } \ \mathcal{A}X = b,\ X \in \mathcal{K}, \qquad \max_{y \in \mathbb{R}^m,\ S \in \mathbb{X}} \langle b, y \rangle \ \text{ s.t. } \ \mathcal{A}^* y + S = C,\ S \in \mathcal{K}^*. \tag{1}$$

Today $\mathcal{K}_\beta \in \{\mathbb{S}^n_+,\ \mathbb{R}^n_+,\ \mathbb{R}^n\}$ (block codes `s`, `l`, `u`), with dual cones $\mathbb{S}^n_+$, $\mathbb{R}^n_+$ and $\{0\}$. PSD blocks are stored in svec form (upper triangle, column by column, off-diagonal entries multiplied by $\sqrt{2}$), so $\langle \cdot, \cdot \rangle$ is the Euclidean inner product on the stacked vector.

3.2 ADMM, sGS-ADMM and their equivalence

ADMM (default). With penalty $\sigma > 0$ and step length $\tau \in (0, 2)$, one iteration is

$$y^{k+1} = (\mathcal{A}\mathcal{A}^*)^{-1}[\sigma^{-1}b - \mathcal{A}(\sigma^{-1}X^k + S^k - C)], \qquad \text{(y-step)} \tag{2}$$

$$S^{k+1} = \Pi_{\mathcal{K}^*}(C - \mathcal{A}^* y^{k+1} - \sigma^{-1}X^k), \qquad \text{(cone step)} \tag{3}$$

$$X^{k+1} = X^k + \tau\sigma(\mathcal{A}^* y^{k+1} + S^{k+1} - C). \qquad \text{(multiplier step)} \tag{4}$$

This is the classical three-step ADMM for SDP [9], exactly (12a)–(12c) of [3] and (24a)–(24c) of [2] with τ, and it is what `SDPSolver::solve` executes today when `switch_admm = 0`.

sGS-ADMM (option). sGS-ADMM [6, 1] inserts a second y-step after the cone step: $y^{k+1/2}$ from (2), S^{k+1} from (3) with $y^{k+1/2}$, then $y^{k+1} = (\mathcal{A}\mathcal{A}^*)^{-1}[\sigma^{-1}b - \mathcal{A}(\sigma^{-1}X^k + S^{k+1} - C)]$, then (4). It costs one extra normal-equation solve per iteration.

Equivalence [3]. Let $P_{\mathcal{A}} = \mathcal{A}^\dagger\mathcal{A}$ with $\mathcal{A}^\dagger = \mathcal{A}^*(\mathcal{A}\mathcal{A}^*)^{-1}$, $P_{\mathcal{A}}^\perp = I - P_{\mathcal{A}}$, and define $z^k := P_{\mathcal{A}}^\perp X^k + \mathcal{A}^\dagger b - \sigma P_{\mathcal{A}} S^k - \sigma P_{\mathcal{A}}^\perp C$. Under $\mathcal{A}\mathcal{A}^* \succ 0$, $\sigma > 0$ and $\tau \in (0, 2)$, both methods generate $z^{k+1} = T(z^k)$ with the same map

$$T(z) = z - P_{\mathcal{A}}[\Pi_{\mathcal{K}}(z) - \mathcal{A}^\dagger b] - \tau P_{\mathcal{A}}^\perp[-\Pi_{\mathcal{K}^*}(-z) + \sigma C] \tag{5}$$

([3, Table 1, eqs. (13) and (21)]). They differ only in how (X, y, S) is recovered from z ([3, eqs. (14) and (22)]), hence in the KKT residuals one observes. The map T is α-averaged with $\alpha = \max\{1, \tau\}/2$ in the metric $G = P_{\mathcal{A}} + \tau^{-1}P_{\mathcal{A}}^\perp$, which is why every $\tau \in (0, 2)$ is admissible; see also [8] and, for convergence of the full sequence, [7].

Consequences for the design

- One iteration engine with an `IterationScheme` strategy (`Admm`, `SgsAdmm`); no compile-time switch.
- τ is a validated user option in $(0, 2)$; presets `tau=1.618` (default) and `tau=1.918` (aggressive). The hidden τ rules of today's sGS branch ($\tau = 1.95$, reduced to $\max(1.618, \tau/1.1)$ when $\eta_d < \text{tol}$) are removed; the value of τ is no longer tied to the scheme.
- The derivations of (13) and (21) in [3] use only the definition of z^k, so a mid-run switch between the two schemes at fixed σ continues the same z-orbit. The "hybrid" mode (sGS for N iterations, then ADMM) is therefore a pure recovery choice and can stay as a cheap option (`scheme_switch_iteration`).

3.3 Cone step for a product cone

The cone step (3) decomposes block by block, $\Pi_{\mathcal{K}^*}(Z) = (\Pi_{\mathcal{K}_1^*}(Z_1), \ldots, \Pi_{\mathcal{K}_B^*}(Z_B))$:

- PSD: $\Pi_{\mathbb{S}^n_+}(Z) = Q\max(\Lambda, 0)Q^\top$ for $Z = Q\Lambda Q^\top$ (self-dual);
- nonnegative orthant: $\max(Z, 0)$ entrywise (self-dual);
- free primal block: $\mathcal{K}^* = \{0\}$, so $S_\beta = 0$.

For cones that are not self-dual, Moreau's decomposition $Z = \Pi_{\mathcal{K}}(Z) - \Pi_{\mathcal{K}^*}(-Z)$ gives $\Pi_{\mathcal{K}^*}(Z) = Z + \Pi_{\mathcal{K}}(-Z)$, so a cone may implement whichever projection is cheaper.

3.4 Scaling and KKT residuals

The solver works on a scaled problem (on by default; Section 11 makes it an option):

1. rows of $\mathcal{A}$ and entries of b are divided by $\nu_i = \max(1, \|A_i\|_2)$, and y_i is multiplied by ν_i;
2. with $b_s = 1 + \|\tilde{b}\|_2$ and $C_s = 1 + \|C\|_2$: $b \leftarrow b/b_s$, $X \leftarrow X/b_s$, $C \leftarrow C/C_s$, $S \leftarrow S/C_s$, $y \leftarrow y/C_s$.

The penalty σ lives in scaled units: on the original data it corresponds to $\sigma\, b_s/C_s$. Termination uses the residuals of [1, eqs. (12)–(13)] in original units,

$$\eta_p = \frac{\|\mathcal{A}X - b\|}{1 + \|b\|}, \quad \eta_d = \frac{\|\mathcal{A}^* y + S - C\|}{1 + \|C\|}, \quad \eta_g = \frac{|\langle C, X\rangle - \langle b, y\rangle|}{1 + |\langle C, X\rangle| + |\langle b, y\rangle|}, \quad \eta = \max(\eta_p, \eta_d, \eta_g),$$

and, at termination and in validation mode, the cone violations of X and S.

3.5 Penalty update (current default)

Each iteration increments w_p if $\eta_p < \eta_d$ and w_d otherwise. At the check iterations $k \in \{k \le 200 : k \equiv 1 \ (10)\} \cup \{200 < k \le 1000 : k \equiv 1 \ (25)\} \cup \{1000 < k \le 5000 : k \equiv 1 \ (50)\} \cup \{5000 < k \le 10^4 : k \equiv 1 \ (200)\} \cup \{k > 10^4 : k \equiv 1 \ (1000)\}$:

$$\begin{aligned} &\text{if } w_p > \max(1,\ 1.35\, w_d): && \sigma \leftarrow \min(\sigma_{\max},\, 2\sigma), \quad w_p \leftarrow 0; \\ &\text{else if } w_d > \max(1,\ 1.35\, w_p): && \sigma \leftarrow \max(\sigma_{\min},\, \sigma/2), \quad w_d \leftarrow 0, \end{aligned}$$

with $[\sigma_{\min}, \sigma_{\max}] = [10^{-3}, 10^3]$. This residual-balancing rule follows the convention of the ADMM phase of SDPNAL+ [10]. Since X is the multiplier of $\mathcal{A}^* y + S = C$, a larger σ weights dual feasibility more. The $\max(1, \cdot)$ guard, the bounds and the removal of the "chasing" monitor (`Monitor1`) are the uncommitted edits of October 13, 2026; before them the guard was absent (so σ always changed at $k = 1$), the bounds were $[10^{-6}, 10^6]$, and after $k = 5000$ the monitor could reset σ to $\max(\sigma, 100)$ or $\min(\sigma, 0.01)$.

4 Assessment of the current code

4.1 What must be preserved

- The numerics are correct: the iteration matches (2)–(4); the three cones project onto the right dual cones; svec conventions, permutations of the CHOLMOD factor, unscaling and the residuals in original units were checked line by line.
- The test suite (87 tests in the plain-ADMM build) generates problems with a known strictly complementary primal–dual solution for PSD, free and nonnegative blocks and for every projection tier, and keeps golden σ histories. On the reference workstation (2× RTX 6000 Ada, CUDA 12.9, SuiteSparse 7.6.1) 85 pass with the October 13 edit; the two failures are expected (B14) or environment-specific (B9).
- Bitwise reproducibility of the exact paths (deterministic `CUSPARSE_SPMV_CSR_ALG2`), the external validator, the certified lower bound, the LOBPCG accept-or-fall-back logic, and the error checks on library calls.

4.2 Structural problems

P1. One class does everything. `SDPSolver` (`include/cuadmm/solver.h`, `src/solver.cu`, 2,229 lines) has over 200 public data members; `init()` is about 590 lines and `solve()` about 1,080. `solve()` mixes the ADMM steps, the three PSD tiers with LOBPCG and composite filtering, the σ rules, termination, external validation, best-iterate tracking, printing, and bookkeeping for a compiled-out mode.

P2. Projection logic at two levels. Large blocks use member functions (`large_full_eig_project`, `large_lobpcg_project`, `large_composite_project`); medium and small blocks are inline in `solve()`. The buffers, handles and streams of all tiers are solver fields.

P3. Configuration in four places. 20 positional parameters of `init()`, 10 of `solve()`, public fields set before either call, and `#define` constants (tier limits 32/1000, LOBPCG thresholds, composite margins). The CLI and MATLAB re-encode defaults independently, which produced B3 and B4.

P4. Dead or compiled-out code. `analyze_blk`, `get_maps`, `MatrixSizes::init` (all superseded by `BlockStructure`), `get_eig_rank_mask`, `max_dense_vector_zero_mask` and the `use_mask` path, `check_gpus`, `bscale2`/`Cscale2`, the disabled monitor, and the sGS code that is compiled out but still threaded through `solve()`.

P5. Scaling in four places. Applied in `init()`, re-applied for warm starts in `solve()`, undone at the end, and undone again on the host in `validate_current_iterate`.

P6. Ad hoc streams. The legacy default stream and non-blocking streams are mixed with manual synchronizations; B2 is a consequence.

P7. Cone-specific branches in seven live files. Conversion kernel, `solver.cu`, external validator, block structure, problem loader, CLI, MATLAB binding. Adding a cone touches all of them.

P8. The PSD projection depends on a submodule whose kernels, helper names and handle conventions differ from the solver's (B1, B10).

4.3 Defect register

Table 2 lists the defects found in the review, with the phase that fixes each one.

Table 2: Defects and inconsistencies of the baseline.

ID	Description	Location / evidence	Phase
B1	`CUBLAS_TENSOR_OP_MATH` on the composite handle equals `CUBLAS_COMPUTE_32F_FAST_16F` for `cublasSgemm`, so the "FP32" composite filter runs at FP16 accuracy. Measured relative error 6.6×10^{-4} / 6.5×10^{-4} instead of 5.3×10^{-6} / 6.5×10^{-6} at $n = 2000/5000$. The test that mirrors the handle accepts 10^{-3}, which hides it.	`src/solver.cu:909`; `test/large_projection_test.hpp:630`	1, 6
B2	Race in `init()`: `get_normA` rewrites `At_csc.vals` on stream 0 while `At_csc` is copied into `A_csr` on non-blocking streams with no ordering. If the copy wins, $\mathcal{A}$ and $\mathcal{A}^*$ are inconsistent.	`src/solver.cu: 535, 541-543`	1

ID	Description	Location / evidence	Phase
B3	Default `switch_proj_max_iter` is 0 in the CLI (low-rank analysis from iteration 2) and 5000 in MATLAB.	`src/main.cu`, `MATLAB/cuadmm_MATLAB.cu`	2
B4	Composite filtering is unreachable: CLI and MATLAB hard-code `EIG_FP64` for both projection phases.	`src/main.cu:651`; `MATLAB/cuadmm_MATLAB.cu:485`	2, 6
B5	`CUADMM_PLAIN_ADMM_ONLY` is defined unconditionally, so sGS-ADMM throws; the README still documents the sGS options.	`CMakeLists.txt:94`	1, 4
B6	`sigscale = max(2 exp(-k/50000), 2)` is always 2 (dead decay).	`src/solver.cu:2059`	4
B7	LOBPCG and Lanczos allocate and free device memory on every call; `cudaFree` synchronizes the device.	`psd_projection/src/lobpcg.cu`, `lanczos.cu`	3, 6
B8	The Lanczos start vector is seeded from `std::random_device` and the clock, so runs with composite filtering are not reproducible.	`psd_projection/src/utils.cu:188`	6
B9	With SuiteSparse 7.6.1, CHOLMOD reports failure on the rank-deficient $\mathcal{A}\mathcal{A}^* + 10^{-15}I$ and `factorize()` throws (`AAtFactorization.RankDeficientRangeSolves` fails); SPOT-type data with dependent rows may fail at setup on such systems.	`include/cuadmm/cholesky_cpu.h:168`	0, 7
B10	Two definitions of the kernel `max_dense_vector_zero_kernel` (cuADMM and submodule).	`src/kernels/dense_scalar.cu`; `psd_projection/src/utils.cu`	6
B11	The composite damping divides the *input* of each polynomial in the code; [2, §3.4] describes rescaling the *output*. Coefficients match [2, Tables 1–2] exactly.	`composite_FP16.cu`, `composite_FP32.cu`	6
B12	Documentation drift: `init()` documents a default σ of `2e2` (signature: 1.0); `lobpcg.h` documents `D` as $m \times m$ (it is a vector); README lists disabled options.	headers, README	1, 10
B13	LOBPCG tolerance is an absolute 10^{-5}, not tied to the stopping tolerance; the acceptance test is necessary but not sufficient (documented in the code).	`src/solver.cu: 36-43, 219-234`	6
B14	The golden test `FixedSigma.LegacyAdaptiveAndPlainAdmmUnchanged` fails after the October 13 penalty edit, by design.	`test/fixed_sigma_validation_test.hpp`	0, 4

5 Design principles

1. **Layers with one-way dependencies.** Front-ends → solver facade → algorithm → components (cones, normal-equation solver, scaling, KKT) → core → CUDA libraries (Figure 1). No component knows the solver.

2. **Strategies, not branches.** Every algorithmic choice (iteration scheme, penalty policy, PSD projector, normal-equation backend, stopping rule) is an object chosen at runtime from options. No compile-time algorithm flags.

3. **One source of truth for options.** Defaults live in `SolverOptions`; the CLI flags and MATLAB fields are generated from, or checked against, the same table.

4. **Explicit GPU resource ownership.** Each component owns its streams, handles and workspace slices, declares cross-stream dependencies with events, and never allocates inside the iteration loop.
5. **Math-to-code traceability.** Names follow Section 3: `y_step`, `cone_step`, `multiplier_step`, `sigma`, `tau` (Appendix B).
6. **Deterministic by default.** Fixed seeds, deterministic SpMV, fixed reduction orders; faster nondeterministic modes are opt-in.
7. **Behavior-preserving migration.** Structural phases must reproduce baseline iterates bit for bit; behavior changes are isolated in labeled phases with tolerance-based acceptance.
8. **Testable components.** Each component can be driven from host data in a unit test without constructing a solver.

6 Target architecture

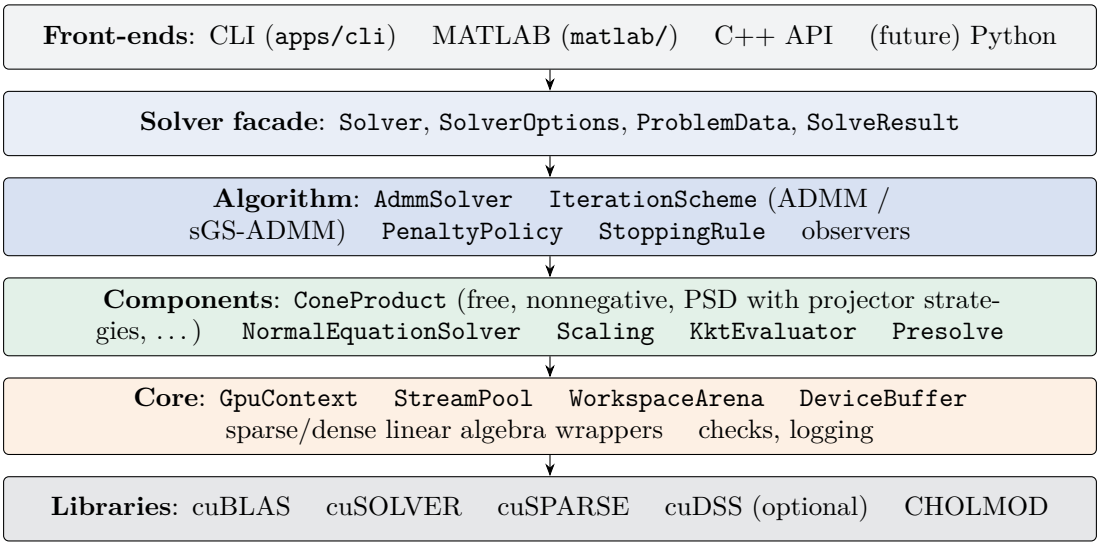


Figure 1: Layers and dependency direction.

6.1 Directory layout

```
cuADMM/
|-- CMakeLists.txt        CUADMM_WITH_{CUDSS,CHOLMOD}, CUADMM_BUILD_{CLI,MATLAB,TESTS,BENCH}
|-- cmake/                FindCUDSS.cmake, FindCHOLMOD.cmake, compiler warnings
|-- include/cuadmm/       public API only
|   |-- cuadmm.h          umbrella header
|   |-- options.h         SolverOptions (single source of defaults)
|   |-- problem.h         ProblemData, ConeSpec
|   |-- result.h          SolveResult, Status, Statistics
|   `-- solver.h          Solver facade
|-- src/
|   |-- core/             GpuContext, StreamPool, WorkspaceArena, buffers, checks, logging
|   |-- linalg/           sparse matrix wrappers (CSR/CSC), SpMV, vector kernels
```

```
|   |-- problem/          BlockLayout, Presolve (dependent rows, grouping by cone), DeviceProblem
|   |-- scaling/          Scaling
|   |-- cone/
|   |   |-- cone.h, cone_product.{h,cu}, cone_registry.{h,cu}
|   |   |-- free_cone.cu, nonnegative_cone.cu        (soc/ added in Phase 9)
|   |   `-- psd/
|   |       |-- psd_cone.{h,cu}, psd_block_group.h, svec.cu
|   |       |-- projectors/   one file per PSD projection strategy (catalog in Section 9)
|   |       |-- policy/       PsdProjectionPolicy, LargeBlockState, RankAnalyzer
|   |       `-- kernels/      composite filters + coefficients, Lanczos bound, LOBPCG
|   |-- linsys/           NormalMatrix, NormalEquationSolver, cudss/ cholmod/ dense/, factory
|   |-- admm/             AdmmState, IterationScheme, PenaltyPolicy, AdmmSolver
|   |-- termination/      KktEvaluator, StoppingRule, BestIterate, ExternalValidator, Certificate
|   |-- io/               TXT problem reader, solution and warm-start I/O
|   `-- report/           IterationObserver: console, CSV, sigma log, JSON, checkpoints
|-- apps/cli/             cuadmm executable (argument parser driven by the option table)
|-- matlab/               MEX gateway and +cuadmm package (options struct)
|-- test/                 unit/, integration/, golden/, data/
|-- bench/                projection and linear-system micro-benchmarks, end-to-end suites
|-- docs/                 this plan, design notes, Doxygen configuration
|-- examples/             unchanged
`-- experiments/          unchanged (frozen records of published runs)
```

6.2 Naming and coding conventions

Item	Convention
Namespaces	`cuadmm`, `cuadmm::core`, `cuadmm::cone`, `cuadmm::cone::psd`, `cuadmm::linsys`, `cuadmm::admm`, `cuadmm::termination`, `cuadmm::report`; implementation details in `detail` or anonymous namespaces.
Files	`snake_case`, named after the main class (`full_evd_projector.cu` defines `FullEvdProjector`); public headers only under `include/cuadmm`.
Types	`PascalCase` classes and structs; `enum class` with `PascalCase` values (`IterationScheme::SgsAdmm`).
Functions, variables	`snake_case`; private members end in `_` (`sigma_`).
Constants	`constexpr` with a `k` prefix (`kDefaultTau`); no `#define` for numerical parameters—those are options.
CUDA kernels	`snake_case_kernel`, file-local (`static` or anonymous namespace), launched only through a host wrapper that takes an `ExecContext`.
Math names	as in Appendix B: `sigma`, `tau`, `y_step`, `cone_step`, `multiplier_step`, `eta_p`, `eta_d`, `eta_g`.
Errors	`CUADMM_CHECK_CUDA/CUBLAS/CUSOLVER/CUSPARSE/CUDSS` macros throwing `cuadmm::Error` with call site and context; no `exit()` in library code.
Ownership	RAII for every CUDA object; non-owning `DeviceSpan<T>` for views; no raw `cudaMalloc` outside `core`.
Style tools	`.clang-format`, `.clang-tidy`, Doxygen comments on every public type; warnings as errors in CI.

6.3 Public API

```
namespace cuadmm {

struct ConeSpec { ConeKind kind; std::int64_t size; double parameter = 0.0; };
```

```
struct ProblemData {                        // host data, original units
  CscMatrix At;                             // vec_len x m, transpose of A in svec form
  SparseVector b, C;
  std::vector<ConeSpec> cones;              // in input order
};

struct WarmStart { std::vector<double> X, y, S; std::optional<double> sigma; };

class Solver {
 public:
  explicit Solver(SolverOptions options);
  void setup(const ProblemData& problem);  // presolve, scale, factorize AA^T, plan cones
  SolveResult solve(const WarmStart* warm_start = nullptr);
  const SolverOptions& options() const;
  void add_observer(std::shared_ptr<report::IterationObserver> observer);
};

}  // namespace cuadmm
```

`SolveResult` carries the unscaled iterate, the residuals η_p, η_d, η_g, cone violations, the certificate quantities, the stop reason, timings per phase and component, and the choices made at setup (normal-equation backend, PSD strategies per block group).

7 Algorithm layer

7.1 The iteration loop

After the reorganization the loop reads as the mathematics:

```
SolveResult AdmmSolver::run(AdmmState& s) {
  for (int k = 1; ; ++k) {
    const KktResiduals r = kkt_.evaluate(s);              // original units, cached
    if (const auto stop = stopping_.check(k - 1, r, s)) return finish(*stop, s);
    scheme_->iterate(s, sigma_, tau_);                     // y-step(s), cone, multiplier
    sigma_ = penalty_->update(k, kkt_.evaluate(s), sigma_);
    observers_.on_iteration(k, s, r, sigma_);
  }
}
```

7.2 Iteration schemes

```
class IterationScheme {
 public:
  virtual ~IterationScheme() = default;
  virtual std::string_view name() const = 0;
  // One iteration of (2)-(4) (ADMM) or its sGS variant.
  virtual void iterate(AdmmState& s, double sigma, double tau) = 0;
  virtual int normal_solves_per_iteration() const = 0;      // 1 for ADMM, 2 for sGS-ADMM
};
class AdmmScheme final : public IterationScheme { /* (2) -> (3) -> (4) */ };
class SgsAdmmScheme final : public IterationScheme { /* (2) -> (3) -> (2') -> (4) */ };
```

Both share the step implementations (`y_step`, `cone_step`, `multiplier_step`) in `AdmmState`; the sGS scheme differs in one extra call. τ is validated in $(0, 2)$ at setup. An optional `scheme_switch_`

`iteration` reproduces the hybrid mode.

7.3 Penalty policies

```
class PenaltyPolicy {
 public:
  virtual ~PenaltyPolicy() = default;
  virtual std::string_view name() const = 0;
  virtual void reset(double sigma0) = 0;
  // Returns sigma for the next iteration; records every change for the sigma log.
  virtual double update(int k, const KktResiduals& r, double sigma) = 0;
};
class FixedPenalty final : public PenaltyPolicy { };              // sigma_k = sigma_0
class ResidualBalancingPenalty final : public PenaltyPolicy { };  // Section 3.5
class LegacyPenalty final : public PenaltyPolicy { };             // old rule, if Q1 keeps it
```

`ResidualBalancingPenalty` exposes the schedule, the factor (2), the ratio (1.35), the $\max(1, \cdot)$ guard and the bounds as options and documents that σ is in scaled units. The `Monitor1` heuristic is deleted; if Q1 keeps a legacy policy, it lives only there.

7.4 Termination and validation

- `KktEvaluator` computes η_p, η_d, η_g once per iteration from the cached residual vectors and the `Scaling` object (no ad hoc unscaling).
- Stopping rules implement `StoppingRule`: the default `InternalKktRule`; the `ValidatedRule`, which is today's validation-aware stopping through the `ExternalValidator`; and the `StrictDimacsRule`. All of them also enforce the iteration and time limits and detect non-finite residuals.
- `BestIterateTracker` keeps the best internal or externally validated iterate.
- `Certificate` computes the cone-measure terms of the certified lower bound through `Cone::measure` (Section 8), so new cones contribute their own terms.

7.5 Reporting

All bookkeeping that today lives in solver fields (`info_*` arrays, `sigma_log`, phase timers, validation history) moves to `IterationObserver` implementations: console table, CSV history, σ log, JSON summary, checkpoint writer. The solver emits events (`on_setup`, `on_iteration`, `on_sigma_change`, `on_projection_switch`, `on_validation`, `on_finish`); observers never influence the iteration.

8 Cones

8.1 Interface

```
enum class ConeKind { Free, Nonnegative, Psd, SecondOrder, RotatedSecondOrder, Exponential, Power
    };

class Cone {
 public:
```

```
  virtual ~Cone() = default;
  virtual ConeKind kind() const = 0;
  virtual std::string_view name() const = 0;
  virtual std::int64_t dimension() const = 0;       // length of this cone's slice
  virtual bool is_self_dual() const = 0;
  virtual WorkspaceRequest workspace() const = 0;   // served by the WorkspaceArena
  // Cone step (3): s = Pi_{K*}(z) on this cone's slice.
  virtual void project_dual(DeviceSpan<const double> z, DeviceSpan<double> s, ExecContext& ctx) =
     0;
  // Optional; default via Moreau: Pi_K(z) = z + Pi_{K*}(-z).
  virtual void project_primal(DeviceSpan<const double> z, DeviceSpan<double> x, ExecContext& ctx);

  // Violation measures and certificate coefficient of X (Side::Primal) or S (Side::Dual).
  virtual ConeMeasures measure(DeviceSpan<const double> v, Side side, ExecContext& ctx) = 0;
  virtual ScalingCompatibility scaling_compatibility() const = 0;
};
```

A `ConeProduct` owns one object per cone kind (each holding all blocks of that kind), maps slices of the vector to cones, runs their `project_dual` on separate streams when profitable, and joins them with events. A `ConeRegistry` maps input codes to factories, so the TXT reader, the MATLAB gateway and the validator never switch on cone letters.

8.2 Existing cones

Code	Primal $\mathcal{K}_\beta$	Dual $\mathcal{K}_\beta^*$	Cone step and status
`s n`	$\mathbb{S}_+^n$	$\mathbb{S}_+^n$	$Q\max(\Lambda,0)Q^\top$ through the strategies of Section 9; correct, tested for all tiers.
`l n`	$\mathbb{R}_+^n$	$\mathbb{R}_+^n$	$\max(z,0)$; correct, tested together with PSD and free blocks (a pure-LP test is added in Phase 5).
`u n`	$\mathbb{R}^n$	$\{0\}$	$s=0$; correct, tested alone and with PSD blocks.

8.3 Adding a cone (checklist)

1. Implement a `Cone` subclass with `project_dual` (or `project_primal` plus Moreau) and `measure`.
2. Register its input code and parameters in `ConeRegistry`.
3. Add a generator case producing a problem with a known strictly complementary solution for the cone (the pattern of `test/regression_problems.hpp`).
4. Add unit tests of the projection: idempotence, Moreau decomposition $z=\Pi_{\mathcal{K}}(z)-\Pi_{\mathcal{K}^*}(-z)$, orthogonality $\langle\Pi_{\mathcal{K}}(z),\Pi_{\mathcal{K}^*}(-z)\rangle=0$, nonexpansiveness, and membership.
5. Declare scaling compatibility and document the cone in the input-format section.

8.4 Planned cones

- **Second-order** $\mathcal{Q}^n=\{(t,x):\|x\|\le t\}$ (code `q`), self-dual, closed form: (t,x) if $\|x\|\le t$; 0 if $\|x\|\le -t$; otherwise $\frac{t+\|x\|}{2}(1,x/\|x\|)$. One batched kernel with a segmented reduction.
- **Rotated second-order** (code `r`) through the orthogonal map to $\mathcal{Q}^n$.

- **Exponential and power cones**: not self-dual, projections by a few Newton steps per cone; the interface already separates primal and dual projections.

The current scaling (row normalization of $\mathcal{A}$ and two scalars) is valid for every cone. Any future variable scaling must be uniform on each PSD and second-order block; `scaling_compatibility()` makes this checkable.

9 PSD cone projection strategies

9.1 Organization

`PsdCone` groups its blocks into `PsdBlockGroups` of equal size n (stored as a contiguous batch of $n \times n$ column-major matrices), converts the svec slice to the batch, projects, and converts back. Each group has a *size class* (small, medium, large) and is assigned a `PsdProjector`; large blocks additionally carry a per-block `LargeBlockState` because their strategy changes during a run.

```
struct PsdBatch { double* data; int n; int count; std::int64_t stride; };  // count n x n
    matrices

enum class ProjectionStatus { Ok, Rejected };  // Rejected: fall back to the exact projector

class PsdProjector {
 public:
  virtual ~PsdProjector() = default;
  virtual std::string_view name() const = 0;
  virtual WorkspaceRequest workspace(int n, int count) const = 0;
  // In place: M_i <- Pi_{S^n_+}(M_i) for every matrix of the batch.
  virtual ProjectionStatus project(PsdBatch batch, ExecContext& ctx) = 0;
};
```

9.2 Strategy catalog

Table 3 lists every projector. All of them exist in some form today; the reorganization gives each one a name, a file, a workspace declaration and a test.

Table 3: PSD projection strategies ("Today" refers to the baseline code).

Projector	Method	Precision	Use, cost and notes
`JacobiBatchedEvdProjector`	`cusolverDnDsyevjBatched`, one call per size group	FP64	Small blocks ($n \leq 32$). Tolerance 10^{-16}, 15 sweeps; `info = n+1` (tolerance not reached) is a warning. Today: small tier.
`BatchedEvdProjector`	`cusolverDnXsyevBatched`, one call per size group	FP64	Medium groups ($32 < n \leq 1000$) with at least two blocks, CUDA $\geq$ 12.8. Verified on the reference workstation up to $n = 1000$: residual $\approx 4 \times 10^{-15}$, orthogonality $\approx 10^{-13}$. Today: medium tier.
`StreamedEvdProjector`	`cusolverDnXsyevd` per block on N streams	FP64	Medium singletons, or every medium block with CUDA $<$ 12.8. One workspace per block. Today: medium tier.

Projector	Method	Precision	Use, cost and notes
`FullEvdProjector`	`cusolverDnXsyevd`, then $Q\max(\Lambda,0)Q^\top$	FP64	Large blocks ($n > 1000$) in the exact phase; also the fallback of every large strategy and the provider of eigenvalues for `RankAnalyzer`. One stream; the final GEMM is batched per size group.
`LobpcgProjector` (`Positive`)	warm-started LOBPCG [14] on Z, $P = V\max(D,0)V^\top$	FP64	Large blocks whose positive rank is below ρn; $k = \lceil 1.2\, r_+ \rceil$ pairs. Cost $O(n^2 k)$ per LOBPCG iteration.
`LobpcgProjector` (`Negative`)	LOBPCG on $-Z$, $P = Z + V\max(D,0)V^\top$	FP64	Large blocks whose negative rank is below ρn (primal low rank). Uses $\Pi(Z) = Z + \Pi(-Z)$.
`CompositeFilterProjector` (`Fp16`, `Fp32`, `Fp32Emulated`)	Lanczos bound, then the composite polynomial filter of [2]	FP16 / FP32 / BF16x9	Large blocks in the warm-up phase; 22 (FP16) or 31 (FP32) GEMMs. Rejected if not finite or if the norm test below fails.
`FullEvdFp32Projector` (optional)	`cusolverDnXsyevd` in FP32	FP32	Experimental; exists in the submodule today, not wired into the solver.

9.3 Selection policy

`PsdProjectionPolicy` assigns projectors from two inputs: the size class (thresholds are options: `psd.small_max = 32`, `psd.medium_max = 1000`, `psd.medium_batch_min = 2`) and, for large blocks, the phase of the run (Figure 2). Unlike today, the warm-up switch and the start of the low-rank analysis are two separate, explicit options; today both hang on `switched_proj_method`, which is why the CLI default of 0 starts the rank analysis at iteration 2.

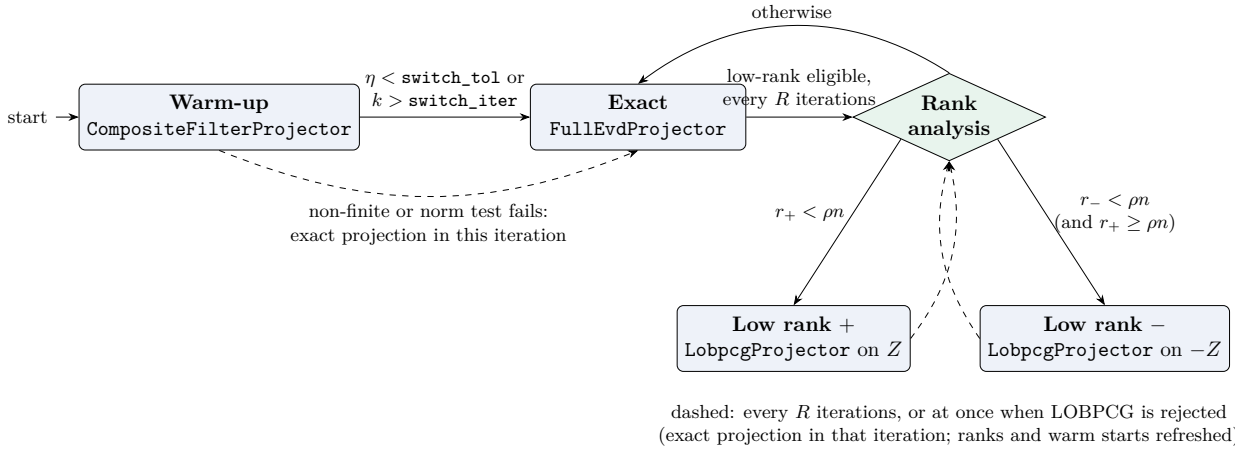


Figure 2: Per-block state machine for large PSD blocks. Without a warm-up method the block starts in **Exact**. Rank analysis is a `FullEvdProjector` call plus `RankAnalyzer`; a rejected LOBPCG result is replaced by an exact projection in the same iteration, which also refreshes ranks and warm starts. Defaults: `switch_tol` $= 10^{-2}$, $\rho = 0.03$, $R = 100$; `switch_iter` is decision Q2.

Rank analysis. With eigenvalues $\lambda_1 \leq \cdots \leq \lambda_n$ of the block,

$$\mathrm{tol}_r = \max\left(10^{-8}\max(|\lambda_1|, |\lambda_n|),\ 10^{-3}\,\mathrm{tol}\right), \qquad r_+ = \#\{\lambda_i > \mathrm{tol}_r\}, \qquad r_- = \#\{\lambda_i < -\mathrm{tol}_r\}.$$

Positive mode is preferred when both ranks are small. The top (or bottom) k eigenpairs are kept as the LOBPCG warm start.

LOBPCG acceptance. A result is accepted if LOBPCG converged, all Ritz values are finite, and the k-th Ritz value is at most tol_r plus the residual norm. This is necessary, not sufficient (Ritz values are lower bounds); the periodic exact analysis and the cone violation reported at termination bound the risk. The tolerance becomes relative to the stopping tolerance (option, B13).

9.4 Integrating the composite polynomial filter

1. **Code move.** The composite kernels (FP16, FP32, FP32-emulated), the Lanczos norm bound (`approximate_two_norm`), LOBPCG and the precision-conversion and symmetrization helpers move to `src/cone/psd/kernels/`. The WIP functions (`compute_eigenpairs`, `*_auto_scale_deflate`) and the commented-out coefficient candidates are not carried over; the standalone repository keeps them, and deflation is recorded as a possible future strategy.
2. **Coefficients as data.** `constexpr` tables `kCompositeFp32[10][3]` and `kCompositeFp16[7][3]` hold the refined coefficients $\tilde{f}$ of [2, Tables 1 and 2] (identical to the code today), obtained by Remez initialization and gradient refinement as in [2], with the Polar Express coefficients [13] kept only as a test baseline, and with a provenance comment. Each step evaluates $aX + bX^3 + cX^5$ with three GEMMs (X^2, X^3, X^3X^2).
3. **Algorithm per block.** Lanczos (20 steps) on Z^2 gives θ and q; $u = \sqrt{\theta + \|Z^2q - \theta q\|}$ [2, Theorem 2]; scale $s = \mu u$ with safety margin $\mu = 1.02$ (FP32) or 1.05 (FP16); $X_0 = Z/s$; $X_t = \tilde{f}_t(X_{t-1}/d_t)$ with damping $d_t = 1.001$ for $t \le 8$ (FP32) and $d_t = 1.01$ for all t (FP16); result $P = s \cdot \frac{1}{2}X_0(I + X_T)$, symmetrized. The damping location (input, as coded, versus output, as in [2, §3.4]) is documented and kept as coded (B11).
4. **Precision and math modes (fixes B1).** One cuBLAS handle per precision mode, each with an explicit setting: FP32 uses `CUBLAS_DEFAULT_MATH`; FP16 uses `cublasGemmEx` with `CUBLAS_COMPUTE_32F_FAST_16F` (unchanged); FP32-emulated uses `CUBLAS_FP32_EMULATED_BF16X9_MATH` with an emulation strategy (to be validated on a Blackwell GPU; the reference workstation cannot run it). `CUBLAS_TENSOR_OP_MATH` is never set.
5. **Safety checks.** Reject (and fall back to `FullEvdProjector`) if the result is not finite or if $\|P\|_F > (1 + 10^{-2})\|Z\|_F$, a free necessary condition since the projection is nonexpansive with $\Pi(0) = 0$. This also catches finite but wrong results from an underestimated scale.
6. **Determinism (fixes B8).** The Lanczos start vector is drawn from a counter-based generator seeded by `(options.seed, block index)`, so runs are reproducible.
7. **Workspace (fixes B7).** $3n^2$ floats (plus $3n^2$ halves for FP16) for the largest large block, and the Lanczos vectors, come from the `WorkspaceArena`; nothing is allocated per call.
8. **Exposure (fixes B4).** Option `psd.large.warmup` $\in$ {`none`, `composite_fp16`, `composite_fp32`, `composite_fp32_emulated`}, available in the CLI (`--psd-warmup`) and MATLAB (`opts.psd.large.warmup`). The default is decision Q3.
9. **Scope.** The filter is used for large blocks only: its GEMM advantage needs n in the thousands [2], while many small blocks are served better by batched EVD. A strided-batched composite filter for medium groups is a benchmark item, not a default.

9.5 LOBPCG improvements

- Preallocate per-block workspaces (B7); keep the eigenpair warm starts in the `LargeBlockState`.
- Reuse ZX_k from the residual computation in the Rayleigh–Ritz step instead of recomputing it inside $Q^\top ZQ$.
- Negative mode: pass $\alpha = -1$ to the GEMMs instead of negating the block in place twice.
- Options for maximum iterations (20), tolerance (relative to tol), ratio ρ (0.03), relaxation (1.2) and re-evaluation period R (100), all reported in the summary.

9.6 PSD options

Option	Default	Today
`psd.small_max`, `psd.medium_max`	32, 1000	`SMALL_MAT_LIMIT`, `MEDIUM_MAT_LIMIT` macros
`psd.medium_batch_min`	2	public field `medium_batch_min_count`
`psd.medium_streams`	15	`--streams`, first MEX argument
`psd.jacobi_tol`, `psd.jacobi_max_sweeps`	10^{-16}, 15	constructor defaults of `BatchEigParameter`
`psd.large.warmup`	Q3	hard-coded `EIG_FP64`
`psd.large.switch_tol`, `psd.large.switch_iter`	10^{-2}, Q2	10^{-2}; CLI 0 / MATLAB 5000
`psd.lowrank.enabled`	true	`--lobpcg`
`psd.lowrank.start_tol`, `psd.lowrank.start_iter`	= switch values	coupled to the warm-up switch
`psd.lowrank.ratio`, `.relaxation`, `.reevaluate`	0.03, 1.2, 100	`LOBPCG_*` macros
`psd.lowrank.max_iter`, `.tol`	20, relative	20, absolute 10^{-5}
`psd.lowrank.rank_rtol`, `.rank_stop_factor`	10^{-8}, 10^{-3}	macros
`psd.composite.margin_fp32`, `.margin_fp16`	1.02, 1.05	`COMPOSITE_*_SCALE_MARGIN` macros
`psd.composite.lanczos_steps`, `.seed`	20, 0	20, random seed

10 Normal-equation solvers

10.1 Role

Every y-step solves $(\mathcal{A}\mathcal{A}^* + \varepsilon I)\, y = r$ with the scaled, row-normalized $\mathcal{A}$; ADMM needs one solve per iteration and sGS-ADMM two. The matrix does not depend on σ, so analysis and factorization run once at setup and the *solve* time dominates. Today CHOLMOD [11] always factors on the CPU (supernodal, falling back to simplicial $LDL^\top$); solves run on the CPU with a device synchronization and two PCIe copies per iteration, unless the factor is dense ($m \geq 2000$, $\mathrm{nnz}(L) > 0.3\, m(m+1)/2$, $8m^2$ bytes $\leq 25\%$ of free memory), in which case L and D are copied to the GPU and each solve is two `cublasDtrsv` calls.

10.2 Interface

```
class NormalEquationSolver {
 public:
```

```
  virtual ~NormalEquationSolver() = default;
  virtual std::string_view name() const = 0;
  virtual SymbolicInfo analyze(const NormalMatrix& AAt) = 0;   // ordering, nnz(L), memory
  virtual FactorInfo factorize(const NormalMatrix& AAt) = 0;   // numeric, pivot statistics
  virtual void solve(DeviceSpan<const double> rhs, DeviceSpan<double> y, ExecContext& ctx) = 0;
  virtual bool solves_on_device() const = 0;
};
```

`NormalMatrix` builds $\mathcal{A}\mathcal{A}^* + \varepsilon I$ once (lower triangle, CSR), on the GPU with `cusparseSpGEMM` or on the host with `cholmod_aat`, and records pivot statistics for diagnostics.

10.3 Backends

`CudssSolver` (preferred). NVIDIA cuDSS [12], distributed separately from the CUDA toolkit; it is NVIDIA's replacement for the sparse Cholesky routines of cuSOLVER, which the CUDA 12.9 headers mark deprecated in favor of cuDSS (`cusolverSpDcsrlsvchol`, `cusolverSpXcsrchol*`). Phases: analysis (reordering, symbolic), factorization, solve, all on the device; matrix type SPD (Cholesky) or symmetric indefinite ($LDL^\top$) for rank-deficient data; lower view; solves on the solver stream with no host synchronization. Enabled by `CUADMM_WITH_CUDSS`; the exact API calls and the minimum version are pinned when the backend is written.

`CholmodSolver` (always available). Today's behavior, moved out of `SDPSolver`: supernodal with simplicial $LDL^\top$ fallback; CPU solves through pinned host buffers and asynchronous copies on the solver stream; the dense-GPU-triangular-solve mode is kept as an option of this backend.

`DenseCusolverSolver` (optional). `cusolverDnXpotrf`/`cusolverDnXpotrs` for SPD, `cusolverDnDsytrf`/`cusolverDnXsytrs` otherwise; or an explicit inverse factor from `cusolverDnXtrtri` so each solve is two triangular matrix–vector products, which parallelize better than `trsv`. Useful when m is a few thousand and the factor is dense.

`PcgSolver` (future). Matrix-free conjugate gradients on $\mathcal{A}\mathcal{A}^*$ with warm start from the previous y, for problems whose factor does not fit; inexact solves are compatible with ADMM when the errors are summable.

10.4 Selection

1. If `linsys.backend` is not `auto`, use it.
2. Otherwise run the symbolic analysis (cuDSS analysis phase, or `cholmod_analyze`) to get $\mathrm{nnz}(L)$ and the memory need.
3. If cuDSS is built in and its factor fits in the memory budget, use `CudssSolver`.
4. Otherwise use `CholmodSolver`, with GPU dense solves if $8m^2$ bytes fit the budget and $\mathrm{nnz}(L) \geq 0.3\, m(m+1)/2$, and CPU solves otherwise.
5. With `linsys.autotune = true`, factor with the two best candidates, time ten solves each and keep the faster. The choice and the timings go to the log and the summary.

10.5 Rank deficiency and presolve

Moment relaxations often have dependent constraints (the SPOT data); a tiny regularization $\varepsilon = 10^{-15}$ plus simplicial $LDL^\top$ is what makes them work today, and it is fragile (B9). A `Presolve`

step detects dependent rows from the factorization pivots (or a sparse QR), removes them after checking that b is consistent, and maps y back at the end. Regularization stays as a fallback option (`linsys.regularization`). cuDSS in SPD mode also requires a nonsingular matrix, so presolve or the $LDL^\top$ mode is needed there too.

10.6 Benchmark protocol

Problems: SPOT (PushBox, PushT, PlanarHand at $N = 1, 10, 30$), pendulum (`licols`), and PLATO / Mittelmann instances including one with a dense factor (`neosfrbr25`). Metrics: analysis, factorization and per-solve time; peak memory; relative residual $\|\mathcal{A}\mathcal{A}^* y - r\|/\|r\|$; end-to-end time to $\eta < 10^{-4}$. GPUs: the RTX 6000 Ada workstation (FP64 throughput is about 1/64 of FP32, which penalizes dense FP64 factorizations and FP64 EVD) and an H100/H200 node. Defaults are set from these results.

11 Scaling

A `Scaling` object holds ν, b_s, C_s and provides `scale_problem`, `scale_iterate`, `unscale_iterate`, `to_original_residuals` and `sigma_to_original`. It is the only code that knows the scaling; the warm start, the termination, the validator and the result use it. Option `scaling.enabled` (default true) allows running unscaled for diagnostics. The unused fields `bscale2`, `Cscale2` are deleted.

12 Core infrastructure

- `GpuContext`: device, library handles per stream role, cuDSS handle, and the math-mode policy of each handle set explicitly at creation.
- **`StreamPool` and `ExecContext`**: named streams (main, small, medium[N], large, linsys, copy). An `ExecContext` bundles a stream, its handles and a workspace slice; cross-stream dependencies use events. Everything runs on non-blocking streams owned by the pool, which removes the class of bugs behind B2.
- `WorkspaceArena`: components declare `WorkspaceRequest`s at setup; the arena allocates once and hands out aligned slices. No `cudaMalloc`/`cudaFree` inside the loop.
- **Buffers**: `DeviceBuffer<T>`, `PinnedHostBuffer<T>`, `DeviceSpan<T>` with `std::size_t` sizes; the overflow checks of `BlockStructure` are kept.
- **Determinism**: option `determinism` $\in$ {`strict` (default), `fast`}.
- **Logging**: a `Logger` with levels; library code does not call `printf`.

13 Unified options

Table 4 is the single table of defaults (PSD options are in Section 9). The CLI flag names are derived from it (`algorithm.tau` becomes `--tau`, with the current flags kept as aliases for one release) and the MATLAB options struct uses the same nested field names.

Table 4: Solver options and defaults ("Q" marks an open decision of Section 16).

Option	Default	Notes
`algorithm.scheme`	`admm`	`admm` or `sgs_admm`; replaces `switch_admm` and the compile flag.
`algorithm.tau`	1.618	Validated in $(0, 2)$; preset `--aggressive` sets 1.918 (Q4).
`algorithm.scheme_switch_iteration`	0 (off)	Hybrid mode: sGS until this iteration, then ADMM.
`algorithm.max_iter`, `.time_limit`	10^6, none	As today in the CLI.
`algorithm.tol`	10^{-4}	Internal stopping tolerance on η.
`penalty.policy`	`residual_balancing`	`fixed`, `residual_balancing`, `legacy` (Q1).
`penalty.sigma0`	100	Scaled units (Section 3.4).
`penalty.sigma_min`, `.sigma_max`	10^{-3}, 10^3	October 13 values.
`penalty.factor`, `.ratio`, `.guard`	2, 1.35, true	The $\max(1, \cdot)$ guard is `guard`.
`penalty.schedule`	as in §3.5	List of (up to iteration, period) pairs.
`scaling.enabled`	true	
`linsys.backend`	`auto`	`auto`, `cudss`, `cholmod`, `dense`.
`linsys.regularization`	10^{-15}	Added to the diagonal of $\mathcal{A}\mathcal{A}^*$.
`linsys.presolve`	true	Remove dependent rows.
`linsys.memory_fraction`, `.autotune`	0.25, false	Budget for dense or cuDSS factors.
`validation.interval`, `.tol`, `.threads`	0, `tol`, 1	Validation-aware stopping as today.
`output.verbosity`, `.history`, `.summary`, `.sigma_log`	1, off, off, off	Observers.
`seed`, `determinism`	0, `strict`	

14 Testing, benchmarking and continuous integration

- **Unit tests** per component: cone properties (idempotence, Moreau decomposition, orthogonality, nonexpansiveness, membership); every PSD projector against an FP64 reference on synthetic spectra (full rank, low rank $\pm$, clustered, tiny n, $n = 1000$ and $n > 1000$), with tolerances per precision (FP32 composite below 10^{-4} median relative error, FP16 composite below 2×10^{-3}; exact projectors near machine precision); normal-equation solvers on full-rank and rank-deficient matrices; penalty policies on scripted residual sequences; option parsing and validation.
- **Integration tests**: generated problems with known optimum for every cone and every projector path, both schemes, $\tau \in \{1.618, 1.918\}$, both normal-equation backends.
- **Golden histories**: per-iteration $(\text{pobj}, \text{dobj}, \eta_p, \eta_d, \sigma)$ of a fixed set of problems, regenerated only by a documented script when a behavior change is intended (B14).
- **End-to-end**: a small SPOT and PLATO subset checked against MOSEK objective values.
- **Benchmarks** (`bench/`): projector micro-benchmarks, normal-equation backends, end-to-end time; results stored per GPU and compared release to release.
- **CI**: build matrix with and without cuDSS and CHOLMOD; CUDA 12.8 and 12.9; nightly `compute-sanitizer` (memcheck, racecheck, synccheck) on the integration tests; GPU runners for Ada and Hopper where available.

15 Migration plan

Each phase is one or a few pull requests on a feature branch, reviewed and merged in order. "Bitwise" means identical per-iteration histories on the golden set and on two real problems.

Phase 0 — Safety net.
Freeze baseline histories (golden set plus one SPOT and one PLATO problem) from `main` with the penalty rule chosen in Q1; add the CI build matrix; make the CHOLMOD rank-deficient test independent of the SuiteSparse version (B9: skip with a message, or presolve later). *Acceptance:* CI green; baseline artifacts stored.

Phase 1 — Cleanup.
Delete dead code (P4); remove `CUADMM_PLAIN_ADMM_ONLY` (sGS back as a runtime option, B5); fix B1 (math mode), B2 (event before the copy), B6, B12; update the README. *Acceptance:* bitwise on the exact paths; composite accuracy test tightened to FP32 levels.

Phase 2 — Options and front-ends.
`SolverOptions`, the `Solver` facade over the existing class, CLI and MATLAB rebuilt on it; resolve B3 and B4 with the Q2/Q3 defaults; old flags as aliases. *Acceptance:* bitwise for equivalent options; option table generated into the documentation.

Phase 3 — Core infrastructure.
`GpuContext`, `StreamPool`, `ExecContext`, `WorkspaceArena`, buffers; LOBPCG and Lanczos stop allocating per call (B7). *Acceptance:* bitwise; no `cudaMalloc` in the loop (checked with Nsight Systems); sanitizers clean.

Phase 4 — Algorithm layer.
`AdmmState`, `IterationScheme` (ADMM, sGS-ADMM), `PenaltyPolicy`, `KktEvaluator`, `StoppingRule`, `BestIterateTracker`, observers; `solve()` becomes `AdmmSolver::run`. *Acceptance:* bitwise for ADMM; sGS-ADMM matches the pre-flag code; τ validation tests.

Phase 5 — Cone layer.
`Cone`, `ConeProduct`, `ConeRegistry`; free and nonnegative cones; the PSD cone wrapping the current tiers unchanged; validator and certificate through `Cone::measure`; pure-LP test. *Acceptance:* bitwise.

Phase 6 — PSD strategies and composite integration.
The projector classes of Table 3, `PsdProjectionPolicy`, `LargeBlockState`, `RankAnalyzer`; submodule code moved in, B8, B10, B11, B13 addressed; submodule removed from `.gitmodules`. *Acceptance:* bitwise on exact and LOBPCG paths; composite accuracy and fallback tests; end-to-end runs with each warm-up method reach the tolerance.

Phase 7 — Normal-equation solvers.
`NormalMatrix`, `NormalEquationSolver`, `CholmodSolver` (bitwise with today), `CudssSolver`, optional `DenseCusolverSolver`, selection and presolve; benchmark report. *Acceptance:* CHOLMOD path bitwise; cuDSS path within 10^{-10} relative residual per solve and same iteration counts within a few percent end to end.

Phase 8 — Data layout.
Permute variables at setup so each cone, and each PSD size group, owns a contiguous slice; run cone projections concurrently on their streams. *Acceptance:* within rounding (summation order changes); speedup measured.

Phase 9 — New cones.
Second-order and rotated second-order cones with generator-based tests; input format and MATLAB `K.q`/`K.r` fields. *Acceptance:* new tests, CBLIB/DIMACS second-order instances solved to tolerance.

Phase 10 — Documentation and release.
README rewrite, Doxygen, examples updated to the new options, migration notes for users of the old MEX signature, tagged release.

Risks and mitigations

- *Silent behavior changes during refactoring* — bitwise acceptance on the golden set for Phases 1–6; any intended change gets its own pull request and regenerated goldens.
- *cuDSS availability and version drift* — optional dependency behind `CUADMM_WITH_CUDSS`; CHOLMOD remains the guaranteed backend; the version is pinned in CMake.
- *Performance regressions from abstraction* — virtual calls happen once per component per iteration, not per element; benchmarks run in CI on every phase.
- *Reproducing published results* — `experiments/` records keep the commit hashes they used; a `legacy` penalty policy is kept if Q1 requires it.

16 Open decisions

Q1. Penalty rule history. Regenerate the golden histories with the October 13 rule, and keep a `legacy` policy (old rule, $[10^{-6}, 10^{6}]$, monitor) only if earlier results must be reproduced from the new code?

Q2. Warm-up switch iteration. Default `switch_iter` of 5000 (MATLAB today) or 0 (CLI today), and should the start of the low-rank analysis be decoupled from it as proposed?

Q3. Default warm-up for large blocks. `none` (today's effective behavior) or `composite_fp16`, as in the warm-start experiments of [2]?

Q4. Default τ. Keep 1.618 or make 1.918 the default after benchmarking.

Q5. Hybrid mode. Keep `scheme_switch_iteration` (cheap given [3]) or drop it.

Q6. MATLAB interface. Keep the positional MEX signature as a compatibility wrapper, or move to `cuadmm.solve(At, b, C, K, opts)` only.

Q7. cuDSS distribution. Minimum version, and whether to use the system, conda or pip package.

Q8. Python bindings. In scope for this reorganization or later?

Q9. Standalone `psd_projection` repository. Keep it for the benchmarks of [2] (frozen), with cuADMM as the maintained home of the kernels?

A Mapping from current files to the new layout

Current	New	Note
`SDPSolver` (`solver.h`, `solver.cu`)	`Solver` facade + `admm/`, `cone/`, `linsys/`, `termination/`, `report/`	split
`SDPSolver::init`	`Solver::setup`: `problem/`, `scaling/`, `linsys/` factory, `ConeProduct` planning	
`SDPSolver::solve` (loop)	`admm/AdmmSolver::run`	
y-step code, `solve_AAt_perm`	`AdmmState::y_step`, `linsys/cholmod/`	
`large_full_eig_project`	`FullEvdProjector` + `RankAnalyzer`	
`large_lobpcg_project`	`LobpcgProjector`	
`large_composite_project`	`CompositeFilterProjector`	
medium and small tiers (inline)	`BatchedEvdProjector`, `StreamedEvdProjector`, `JacobiBatchedEvdProjector`	
`cone_measures`, `certified_lower_bound`	`Cone::measure`, `termination/Certificate`	
`validate_current_iterate`, `external_validation.*`	`termination/ExternalValidator`, `ValidatedRule`	
σ update in `solve`, `monitors.h` best-iterate fields	`admm/PenaltyPolicy` `BestIterateTracker`	monitor deleted
`info_*` arrays, `sigma_log`, phase timers	`report/` observers	
`memory.h`	`core/device_buffer.h`, `core/handles.h`, `linalg/sparse_matrix.h`	
`check.h`	`core/check.h`	
`cublas.h`, `cusparse.h`, `cusolver.h`	`linalg/` wrappers, projector files	
`kernels/*.cu`	`linalg/vector_ops.cu`, `cone/psd/svec.cu`, `linsys/permutation.cu`	
`kernels/type_conversion.cu`	`matlab/`	MEX index conversion
`block_structure.*`, `matrix_sizes.*`	`problem/BlockLayout`, `cone/psd/PsdBlockGroup`	
`utils/analyze_blk.cu`, `utils/get_maps.cu`, `MatrixSizes::init`, `utils/get_eig_rank_mask.cu`, `utils/check_gpus.cu`, `max_dense_vector_zero_mask`	—	deleted (unused)
`rank.cu`	`cone/psd/policy/RankAnalyzer`	
`cholesky_cpu.h`	`linsys/cholmod/CholmodSolver`	
`eig_cpu.h`	`test/` helper	test-only
`io.*`, `problem.*`	`io/`, `problem/`	
`main.cu`	`apps/cli/`	
`MATLAB/cuadmm_MATLAB.cu`	`matlab/`	
`psd_projection/src/composite_FP16.cu`, `composite_FP32.cu`, `composite_FP32_emulated.cu`	`cone/psd/kernels/composite_*.cu`	
`psd_projection/src/lanczos.cu`	`cone/psd/kernels/lanczos_bound.cu`	only the norm bound
`psd_projection/src/lobpcg.cu`	`cone/psd/kernels/lobpcg.cu`	

Current	New	Note
`psd_projection/src/utils.cu`	`cone/psd/kernels/precision.cu`, `symmetrize.cu`	
`psd_projection/src/eig_FP64_psd.cu`, `eig_FP32_psd.cu`	`FullEvdFp32Projector` (optional)	
`psd_projection/test/*`	`test/unit/psd/`	

B Notation and code names

Symbol	Meaning	Code name
X, y, S	primal variable, dual multiplier, dual slack	`X`, `y`, `S`
$\mathcal{A}$, $\mathcal{A}^*$	constraint map and adjoint	`A`, `At`
$\mathcal{A}\mathcal{A}^*$	normal matrix	`NormalMatrix`
$\mathcal{K}$, $\mathcal{K}^*$	cone and dual cone	`Cone`, `project_dual`
σ, τ	penalty, step length	`sigma`, `tau`
(2), (3), (4)	the three steps	`y_step`, `cone_step`, `multiplier_step`
$\eta_p, \eta_d, \eta_g, \eta$	residuals	`eta_p`, `eta_d`, `eta_g`, `eta`
ν, b_s, C_s	scaling factors	`Scaling::row_norms`, `b_scale`, `c_scale`
r_+, r_-, tol_r	ranks and rank tolerance	`RankAnalyzer`
ρ, R	low-rank ratio, re-evaluation period	`psd.lowrank.ratio`, `.reevaluate`

References

[1] A. Groudiev, S. Kang, and H. Yang. cuADMM: GPU-accelerated first-order optimization for large-scale multi-block semidefinite programs. In *RSS Workshop on Fast Motion Planning and Control in the Era of Parallelism*, 2025. https://agroudiev.github.io/assets/pdf/8_cuADMM_GPU_Accelerated_First.pdf.

[2] S. Kang, H. Han, A. Groudiev, and H. Yang. Factorization-free orthogonal projection onto the positive semidefinite cone with composite polynomial filtering. arXiv:2507.09165, 2025 (v2, 2026).

[3] S. Kang and H. Yang. Afterimage slow regions in first-order methods for linear conic programming. arXiv:2609.09449, 2026.

[4] S. Kang, X. Xu, J. Sarva, L. Liang, and H. Yang. Fast and certifiable trajectory optimization. In *International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2024. arXiv:2406.05846.

[5] S. Kang, X. Jiang, and H. Yang. Local linear convergence of the alternating direction method of multipliers for semidefinite programming under strict complementarity. arXiv:2503.20142, 2025.

[6] L. Chen, D. Sun, and K.-C. Toh. An efficient inexact symmetric Gauss–Seidel based majorized ADMM for high-dimensional convex composite conic programming. *Mathematical Programming*, 161(1–2):237–270, 2017.

[7] L. Chen, X. Li, D. Sun, and K.-C. Toh. On the equivalence of inexact proximal ALM and ADMM for a class of convex composite programming. *Mathematical Programming*, 185(1):111–161, 2021.

[8] D. Gabay and B. Mercier. A dual algorithm for the solution of nonlinear variational problems via finite element approximation. *Computers & Mathematics with Applications*, 2(1):17–40, 1976.

[9] Z. Wen, D. Goldfarb, and W. Yin. Alternating direction augmented Lagrangian methods for semidefinite programming. *Mathematical Programming Computation*, 2(3):203–230, 2010.

[10] D. Sun, K.-C. Toh, Y. Yuan, and X.-Y. Zhao. SDPNAL+: A Matlab software for semidefinite programming with bound constraints (version 1.0). *Optimization Methods and Software*, 35(1):87–115, 2020.

[11] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software*, 35(3), 2008.

[12] NVIDIA Corporation. cuDSS: NVIDIA CUDA direct sparse solver library, documentation. https://docs.nvidia.com/cuda/cudss/.

[13] N. Amsel, D. Persson, C. Musco, and R. Gower. The Polar Express: optimal matrix sign methods and their application to the Muon algorithm. arXiv:2505.16932, 2025.

[14] A. V. Knyazev. Toward the optimal preconditioned eigensolver: locally optimal block preconditioned conjugate gradient method. *SIAM Journal on Scientific Computing*, 23(2):517–541, 2001.